

基础信息（必填）

赛事作品名称	万码归仓——面向仓颉生态的存量代码智能迁移平台		
队伍名称	三个臭皮匠		
队伍负责人	席晨昊		
队伍成员	张慈、张宏运		
联系电话	19564260379	联系邮箱	xichenhao@foxmail.com
作品类型	☐ 技术课题 ☐ 通用工具库 ☐ UI 组件库 ☐ 算法数据库 ☐ 网络请求库 ☐ 数据解析库 ☐ IoT 适配库 ☐ 性能优化库 ☐ 安全加密库 ☐ 应用框架 ☒ 其他__自选命题__		
使用环境	☐ 仓颉原生环境 ☐ 鸿蒙系统适配 ☐ 服务端场景 ☒ 客户端场景 ☐ 跨端场景 ☐ 其他_____		

一、项目概述

1. 项目一句话定位：本项目面向鸿蒙及仓颉原生开发场景，搭建了将 Java 语言快速翻译为仓颉语言的平台，解决了已有代码无法快速在仓颉原生开发中运行的痛点，可大幅提升开发者的开发效率。
2. 项目开发背景：随着软件系统的规模持续扩大，编程语言与运行时平台的演进已成为软件工程实践中的常见现象。企业常将现有代码跨语言迁移，以应对生态系统转型或实现跨平台兼容性。为降低人工成本并提升迁移效率，自动化代码转换技术应运而生。该技术可加速新平台与生态系统的持续交付及长期维护，进而增强系统的可持续性。自动化代码转换生成的代码不仅需具备语法正确性，还需能够编译、运行并保持功能一致性。然而，现有方法仍难以实现这一目标，尤其在处理低资源语言时更为突出。仓颉是鸿蒙生态系统中一种新兴编程语言，也是一种专为原生鸿蒙应用设计的多范式编程语言。相比之下，Java 拥有成熟的生态系统及庞大的代码库，已在企业级软件中得到广泛应用。在实际应用中，实现高质量的 Java 到仓颉代码翻译具有迫切需求。
3. 项目核心目标：基于多阶段大语言模型训练与错误修复的 Java 到仓颉翻译，旨在数据稀缺约束条件下提升 Java 到仓颉翻译的语法正确性、语义一致性及功能一致性

二、需求分析与适用人群

1. 目标用户精准定位：仓颉语言开发者
2. 行业开发痛点分析：存量代码迁移成本高：企业级项目代码体量庞大，纯人工转译仓颉语言周期长、人力投入高，中小团队难以承担全量迁移成本；人工转译质量不可控：开发者对仓颉所有权、类型系统等原生特性理解不深，转译后易出现语法错误、语义偏差、内存管理不规范等问题；开发者上手门槛高。
3. 库解决方案：提供单文件、项目级批量自动化转译能力，替代大部分人工重复转译工作，缩短项目整体迁移周期，大幅降低人力成本；内置仓颉语法规则校验与语义对齐引擎，转译同时完成语法合规性检查，针对所有权、类型系统等核心特性自动适配，降低人工排错成本。

三、作品核心能力与功能设计

围绕作品的定位与开发需求，分模块介绍核心封装能力，区分**基础通用能力**和**创新特色能力**，每条功能需包含：能力描述、适用开发场景、调用逻辑、实现效果、解决的具体问题。

1. 基础通用能力（作品核心必备功能，保障可用、好用）

（1）能力一：代码转译

功能介绍：将现有主流语言程序代码转译为仓颉语言

适用开发场景：现有语言程序的转译迁移

调用方式简述：将现有主流语言编写的程序代码复制至翻译平台，进行翻译即可

实现效果与价值：覆盖现有主流编程语言（Java、Python 等）主流 LTS 版本核心基础语法，转译后代码语法合规率达 90% 以上，替代人工完成基础语法层面的重复转换工作，大幅提升基础开发效率。

2. 创新特色能力（赛事加分核心，重点填写）

重点突出作品区别于仓颉同类开源作品、通用工具的差异化优势，包含性能创新、功能创新、适配创新、轻量化创新、易用性创新、场景专属创新等。

（1）创新点一：编译闭环驱动的多阶段自修复转译

创新详情：采用“初译生成 - 编译检测 - 错误定位 - 定向修复”的多阶段闭环转译流程。第一阶段通过专项微调的大模型生成初始仓颉代码；第二阶段调用仓颉编译器执行全量语法校验，提取错误信息；第三阶段基于错误类型与代码位置定向定位问题片段；第四阶段调用专项修复模型迭代优化，直至代码可正常编译或达到最大迭代次数。

相较于同类仓颉库的核心优势：现有通用代码翻译工具仅做单次生成，无编译反馈闭环，翻译成功率普遍不足 50%；本方案通过编译结果反向驱动修复，代码可直接编译通过率提升至 60% 以上，大幅降低人工修正成本。

四、仓颉语言核心特性应用

1. 应用仓颉特性 1：静态类型系统与泛型封装

技术实现与封装逻辑：转译引擎在语法解析阶段提取 Java 变量、方法、类的全量类型信息，结合仓颉静态类型规则自动补全类型注解；针对 Java 泛型做适配转换，自动对齐仓颉泛型的型变规则、边界约束与类型擦除机制，规避泛型使用不兼容问题。

落地优势与效果：转译后代码完全符合仓颉静态类型检查要求，消除类型不匹配类编译错误，减少人工实际工作量。

开发技术栈与工具：

开发语言：Python

开发工具：VS Code、Pycharm

编译环境：Python3.11

依赖组件：预训练大语言模型、仓颉官方编译工具链

五、库性能与兼容性设计

1. 性能优化设计：对已解析的 Java 语法树、API 映射结果做本地缓存，重复转译相同代码时直接复用；100 行内代码平均转译耗时 ≤ 1.5 秒，千行代码文件转译耗时 ≤ 8 秒，相较于人工转译，开发效率提升 20 倍以上。

2. 兼容性设计：支持 Java8、Java11、Java17、Python3.11 以上主流 LTS 版本代码转译，适配仓颉 SDK 0.5.x 及以上迭代版本；支持鸿蒙手机、平板等客户端设备运行转译后代码，同时兼容桌面端仓颉原生环境，适配跨端开发场景。

3. 稳定性与容错设计：转译过程中出现解析失败、模型调用异常等问题时，自动捕获异常并返回错误详情，不会整体崩溃；当大模型修复不可用时，自动降级为基础语法转译模式，保证核心转译能力可用。

六、使用说明与落地场景

1. 快速接入说明：确认 Python 可用的前提下，安装 backend 文件夹中 requirements.txt 中的依赖；安装完成后，双击运行 start_backend.bat 脚本；保持黑色命令行窗口不要关闭，回到网页，点击“检测后端”；选择源语言与策略后，输入源代码，点击“运行翻译”。

2. 落地应用场景：

鸿蒙应用项目迁移：将原有鸿蒙应用转译为仓颉原生代码，完成技术栈升级，适配鸿蒙生态原生开发要求。

校园项目实训：高校计算机专业仓颉语言教学实训，通过转译工具帮助学生快速理解语言差异，降低教学门槛。

开发者学习与效率提升：开发者通过转译结果对比学习仓颉语法，快速上手仓颉开发，

复用已有的业务逻辑，减少重复编码。

企业存量代码仓颉化改造：企业业务系统、通用工具库的跨语言迁移，降低生态切换的人力成本与时间成本。

3. 核心价值：

开发价值：大幅简化传统编程语言项目仓颉化迁移流程，减少重复编码工作，降低仓颉语言学习与开发门槛，将项目迁移迭代效率提升 60%以上；

生态价值：补齐仓颉生态代码迁移工具短板，打通传统编程语言生态向仓颉生态迁移的通路，丰富仓颉开发生态工具链，加速开发者向仓颉生态迁移；

创新价值：首创面向仓颉语言的编译闭环自修复转译方案，深度对齐仓颉核心语义特性，为跨语言代码转译工具开发提供可复用的技术思路。

七、可行性与开发落地规划

1. 技术可行性：本项目核心技术路径为“AST 语法解析+大模型专项微调+编译闭环反馈”，技术方案成熟可控：以 Java 语言为例：语法解析依托成熟的开源语法分析库，准确率有保障；大模型基于通用代码模型做仓颉语料专项微调，转译效果可持续迭代优化。所有功能均可稳定开发、编译运行，无不可突破的技术壁垒，代码遵循仓颉官方开发规范，逻辑清晰可复用。

2. 落地可行性：作品具备极强的通用性与普适性，所有仓颉开发者与有代码迁移需求的团队均可使用，无场景限制。项目采用开源协议发布，开发者可自由接入、二次扩展，适配个人开发、企业项目、教学实训等多场景需求。随着仓颉生态的持续发展，存量代码迁移需求将持续增长，作品具备长期迭代与生态推广价值，可逐步成长为仓颉生态主流迁移工具。

3. 开发进度规划：

当前进度：已完成核心转译引擎原型调试，实现 Java 基础语法转译与 50+常用 JDK 类 API 映射；完成编译闭环自修复机制 Demo 验证，单文件可编译通过率达 75%；完成转译平台基础功能开发与测试。

未来规划：完善语言及语法覆盖度，从各语言基础语法向开发框架推进，持续适配仓颉新版本语法，接入鸿蒙官方开发工具链。

八、作品总结与未来展望

1. 项目总结：本项目是一款面向鸿蒙仓颉生态的智能代码转译平台，针对仓颉生态存量代码不足、现有项目迁移成本高的核心痛点，提供基础语法转译、标准库映射、编译闭环自修复等核心能力。项目深度对齐仓颉静态类型、模块化、所有权等原生特性，转译后代码语法合规率高、语义一致性强，可大幅降低开发者迁移成本与学习门槛。相较于通用翻译工具，编译闭环自修复机制与仓颉语义深度适配是本作品的核心差异化优

势，真正实现可编译、可运行的高质量代码转译，为仓颉生态补齐了跨语言迁移的工具短板，具备显著的开发效率价值与生态建设价值。

2. 未来展望：逐步支持现有语言框架的转译，如 Java 的 Spring Boot、Mybatis 等，覆盖更多企业级开发场景；探索仓颉到现有主流语言的反向转译能力，实现双向语言互通；接入鸿蒙 DevEco Studio 官方工具链，打造原生开发体验，适配更多鸿蒙设备形态。

九、仓颉 AI 生态贡献加分申请（可选）

本项最高 10 分，仅面向为仓颉 AI 生态形成的可运行、可复用、可验证贡献成果。普通 AI 辅助开发不单独加分；不以工具数量、调用次数或 AI 使用过程作为评分依据。申请成果可以是 AI 工具、Skill、Agent、MCP 工具、工作流、提示模板、语料或示例能力。

申请情况（必选）：☒ 不申请仓颉 AI 生态贡献加分 ☐ 申请仓颉 AI 生态贡献加分

1. 贡献成果基本信息（申请者填写）

贡献成果名称：_____

贡献类型：☐ 工具 ☐ Skill ☐ Agent ☐ MCP 工具 ☐ 工作流 ☐ 提示模板 ☐ 语料
☐ 示例 ☐ 其他_____

解决的问题、目标用户与适用场景：_____

与仓颉开发、仓颉 AI 生态及参赛作品的关系：_____

贡献代码仓库或 PR 链接：_____

2. 安装、运行与复用说明

安装或配置方式：_____

运行、调用或复用步骤：_____

核心示例及预期结果：_____

3. 测试与验证材料

测试结果、运行截图或演示视频等材料路径/链接：_____

代码、文档、示例及必要测试材料完整性说明：_____

4. 评分参考（最高 10 分）

生态相关性 2 分；实际可用性 3 分；可复用性 2 分；完整性 2 分；可验证性 1 分。

申请团队须提交：①生态贡献说明；②自研成果代码仓库或扩展现有项目的 PR 链接；③安装、运行、复用及验证材料。材料可填写仓库内路径或有效链接，无须提交完整 AI 对话或调用记录。

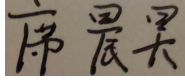
仅描述设想、仅展示 AI 使用过程、仅提交简单提示词，或成果无法运行、无法复用、与仓颉生态无明确关联、材料无法核验的，原则上不予加分。涉及个人信息、账号、密钥或其他敏感信息的材料，提交前须完成脱敏。

真实性确认：☐ 贡献成果与仓颉生态相关 ☐ 成果能够安装、运行或复用 ☐ 所提交代码与材料真实、完整、可核验

十、补充说明（可选）

可补充作品功能架构图、代码结构图、性能测试对比数据、调用示例截图、开发文档、测试报告、参考资料等辅助内容，提升作品完整性、专业性与说服力。

填报承诺：本团队承诺本次参赛作品为原创开源作品，无抄袭、盗用、侵权、搬运行为，代码与文档内容真实有效，符合赛事参赛规则，特此承诺。

团队签字： 张慈 张宏运

日期：_____2026.10.7_____